

DevOps & Cloud Engineer Interview Questions & Answers

Kubernetes • Docker • CI/CD & Jenkins • Terraform • AWS

Observability • Security • Deployment Strategies • SRE • Python

by
Sahil Kumar Sahu

Contents

Kubernetes — Core Concepts	3
Kubernetes — Scheduling & Internals	4
Kubernetes — Networking & DNS	6
Kubernetes — Troubleshooting Scenarios	8
Kubernetes — Resources, Security & Operations	9
Docker & Containers	10
CI/CD & Jenkins	12
Terraform & Infrastructure as Code	14
AWS & Cloud	18
Networking & System Design	22
Observability, Monitoring & Logging	23
Security & Secrets Management	25
Deployment Strategies	26
GitOps, ArgoCD & Git	27
Incident Response, SRE & Reliability	28
Python (Scripting for DevOps)	30

Kubernetes — Core Concepts

Q1. What is a pod and why is it the smallest deployable unit in Kubernetes?

A pod is one or more tightly-coupled containers that share a network namespace (same IP and port space), storage volumes, and lifecycle. It is the smallest unit because Kubernetes schedules and manages whole pods, not individual containers — containers in a pod always run on the same node and can talk over localhost. Most pods run a single container; multi-container pods are used for sidecar patterns (logging agents, proxies).

Q2. Difference between Deployment, StatefulSet and DaemonSet?

A Deployment manages stateless, interchangeable pods with random names and supports rolling updates and easy scaling. A StatefulSet gives pods stable, ordered identities (pod-0, pod-1), stable network names, and persistent per-pod storage — used for databases, Kafka, etc. A DaemonSet runs exactly one pod on every (or selected) node, used for node-level agents like log collectors, CNI plugins, or monitoring exporters.

Q3. What is the difference between ConfigMap and Secret?

Both inject configuration into pods as env vars or mounted files, but a ConfigMap holds non-sensitive plain-text data, while a Secret holds sensitive data (passwords, tokens, keys) and is stored base64-encoded with the option of encryption at rest in etcd. Secrets also support tighter RBAC and integration with external stores. Note: base64 is encoding, not encryption — secrets still need at-rest encryption and RBAC to be truly secure.

Q4. What are liveness and readiness probes and how do they work?

A liveness probe tells the kubelet whether the container is healthy; if it fails repeatedly, the kubelet restarts the container. A readiness probe tells whether the pod is ready to receive traffic; if it fails, the pod is removed from Service endpoints but not restarted. Both can be HTTP, TCP, or exec checks with configurable initialDelaySeconds, periodSeconds, and failureThreshold. A startup probe can also be used for slow-starting apps.

Q5. What happens if a readiness probe fails?

The pod's IP is removed from the Endpoints/EndpointSlice of any Service that selects it, so it stops receiving new traffic, but the container keeps running and is not restarted. Once the probe passes again the pod is re-added to the rotation. This is the mechanism that lets a pod gracefully drop out during temporary overload or while warming up.

Q6. What is CrashLoopBackOff and how do you debug it?

CrashLoopBackOff means a container keeps crashing and Kubernetes is backing off (waiting increasing intervals) before restarting it. Debug by checking `kubectl describe pod` for the exit code and events, then `kubectl logs <pod> --previous` for the crashed instance's logs. Common causes: app error/misconfig, missing env var or config, failing liveness probe, OOMKill (exit 137), bad image command, or a dependency (DB) not reachable.

Q7. What does Evicted state mean, and how does it differ from OOMKilled?

Eviction is a node-level action: the kubelet evicts pods when the node is under resource pressure (memory, disk, PIDs) to reclaim resources, deleting the pod. OOMKilled is a container-level action: the Linux kernel's OOM killer kills a single container that exceeded its memory limit (exit code 137).

Eviction removes the whole pod from the node; OOMKill just kills and restarts the offending container within the pod.

Q8. How does Kubernetes decide which pod to evict?

Under node memory pressure, the kubelet ranks pods by QoS class and how far they exceed their requests. BestEffort pods (no requests/limits) are evicted first, then Burstable pods exceeding their requests, and Guaranteed pods last. Within a class, pods using the most resource above their request are evicted first. Priority/PriorityClass also factors in.

Q9. What are QoS classes (BestEffort, Burstable, Guaranteed)?

Guaranteed: every container has equal requests and limits for both CPU and memory — highest protection. Burstable: at least one container has a request but they aren't all equal to limits — medium protection. BestEffort: no requests or limits set — first to be evicted/killed under pressure. QoS drives eviction order and OOM scoring.

Q10. What is Exit Code 137 and what causes it?

Exit code 137 = 128 + 9, meaning the process was terminated by SIGKILL (signal 9). In Kubernetes it almost always means the container was OOMKilled — it exceeded its memory limit and the kernel killed it. It can also occur if a node forcibly kills a container during shutdown. Fix by raising memory limits, fixing leaks, or profiling memory usage.

Q11. Can a pod move to another node automatically? What is the difference between pod restart and pod recreation?

A running pod is never live-migrated; if the node fails or the pod is evicted/deleted, the controller (Deployment/ReplicaSet) creates a new pod, possibly on another node. A pod restart is the kubelet restarting a container inside the same pod (same pod name/UID, restart count increments). Pod recreation is a brand-new pod object with a new name/UID/IP, done by a controller. Bare pods (no controller) do not get rescheduled.

Q12. What is the default DNS policy in Kubernetes?

The default dnsPolicy for pods is `ClusterFirst`, meaning DNS queries first go to the cluster DNS (CoreDNS); names not matching the cluster domain are forwarded to the upstream resolver. Other policies are `Default` (inherit the node's resolv.conf), `None` (fully custom via dnsConfig), and `ClusterFirstWithHostNet` (for host-network pods that still need cluster DNS).

Kubernetes — Scheduling & Internals

Q13. How does the Kubernetes scheduler decide which node to place a pod on?

The scheduler runs a two-phase process: filtering (predicates) eliminates nodes that can't run the pod — insufficient CPU/memory, taints not tolerated, node selectors, affinity rules, volume constraints — then scoring (priorities) ranks the surviving nodes by spread, resource balance, affinity preferences, and image locality. The highest-scoring node wins, and the pod is bound to it. The kubelet on that node then actually starts the pod.

Q14. Why would a pod remain in Pending state?

Pending means the scheduler hasn't been able to bind the pod to a node. Common causes: no node has enough free CPU/memory to satisfy requests, taints without matching tolerations, node selectors/affinity that no node satisfies, unbound PersistentVolumeClaims, or the cluster being at capacity (which may trigger the Cluster Autoscaler). `kubectl describe pod` shows the scheduling failure reason in events.

Q15. What happens internally when you run `kubectl apply`?

kubectl builds the desired object, computes a three-way merge against the last-applied-configuration annotation, and sends it to the API server. The API server authenticates, authorizes (RBAC), runs admission controllers/webhooks, validates the schema, and persists the object in etcd. The relevant controller (via its watch) notices the change and reconciles actual state toward desired state — e.g., a Deployment controller updates a ReplicaSet, which creates pods the scheduler then places.

Q16. What happens inside the kubelet when a pod is created?

The kubelet watches the API server for pods bound to its node. On a new pod it: pulls images via the container runtime (CRI), sets up the pod sandbox and network via the CNI plugin, mounts volumes via CSI, applies cgroup limits, and starts containers in order (init containers first). It then runs probes and continuously reports pod status back to the API server.

Q17. How does Kubernetes handle pod failures and self-healing?

Controllers continuously reconcile desired vs actual state. If a container crashes, the kubelet restarts it per restartPolicy. If a pod is deleted or a node goes NotReady, the ReplicaSet/Deployment notices the replica count is below desired and creates a replacement pod. Node-level failures trigger the node controller to mark pods for deletion after a timeout so they're rescheduled elsewhere. Probes drive restart and traffic decisions.

Q18. What happens when a node goes NotReady?

The kubelet stops sending healthy heartbeats, so the node controller marks the node NotReady. After the pod-eviction-timeout (default ~5 min), pods on that node are marked for deletion and their controllers recreate them on healthy nodes. Pods with local storage or that are part of StatefulSets may need manual intervention. The node is cordoned from new scheduling until it recovers.

Q19. How does autoscaling work — how does the Horizontal Pod Autoscaler (HPA) make decisions?

The HPA controller periodically reads metrics (CPU/memory from metrics-server, or custom/external metrics) and computes $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times \text{currentMetric} / \text{targetMetric})$. It scales up quickly and scales down conservatively (with stabilization windows) to avoid flapping. It only adjusts replica counts within configured min/max; it does not change resource requests (that's the VPA).

Q20. How do HPA and Cluster Autoscaler work together under high load?

The HPA adds more pod replicas to handle load. If the cluster lacks node capacity to schedule those new pods, they go Pending, which signals the Cluster Autoscaler to add nodes to the node group. When load drops, the HPA scales pods down, and the Cluster Autoscaler removes

underutilized nodes after consolidating pods. They operate at different layers — pods vs nodes — and complement each other.

Q21. How does service routing work at the iptables level / what is the role of kube-proxy?

kube-proxy runs on every node and programs the dataplane so that traffic to a Service's ClusterIP is load-balanced to backend pod IPs. In iptables mode it writes NAT rules that randomly DNAT the virtual ClusterIP to one of the endpoint pod IPs. In IPVS mode it uses kernel IPVS for more efficient load balancing at scale. kube-proxy watches Services/EndpointSlices and keeps these rules in sync.

Kubernetes — Networking & DNS

Q22. How does DNS work inside Kubernetes and what is CoreDNS?

CoreDNS runs as a Deployment and is exposed as a ClusterIP Service; the kubelet injects its address into each pod's `/etc/resolv.conf`. Pods resolve Services by name — ``svc.namespace.svc.cluster.domain`` — and CoreDNS answers from records it builds by watching the API server. Headless services resolve to individual pod IPs. Troubleshoot CoreDNS by checking its pods/logs, the kube-dns Service, and `resolv.conf` inside a pod.

Q23. How do you debug DNS resolution failing in a pod / in the cluster?

Exec into a pod and test resolution (``nslookup kubernetes.default``, then an external name). Check `/etc/resolv.conf` points to the cluster DNS IP. Verify CoreDNS pods are Running and read their logs; check the kube-dns Service and its endpoints. Confirm NetworkPolicies aren't blocking UDP/TCP 53, and that nodes can reach CoreDNS. Also check CoreDNS resource limits — under load it can be throttled or OOMKilled.

Q24. What is ClusterIP, NodePort and LoadBalancer?

ClusterIP is the default — a stable virtual IP reachable only inside the cluster. NodePort exposes the Service on a static port on every node's IP, reachable from outside the cluster. LoadBalancer provisions an external cloud load balancer (e.g., AWS ELB/NLB) that forwards to the NodePort/pods. They layer: LoadBalancer builds on NodePort, which builds on ClusterIP.

Q25. How do you expose Kubernetes apps to external traffic?

Options: a Service of type LoadBalancer (one cloud LB per service), NodePort (raw node port), or — most commonly — an Ingress controller (e.g., NGINX, ALB Ingress) that terminates TLS and routes HTTP(S) by host/path to multiple services behind a single LB. The newer Gateway API is the successor to Ingress for more advanced routing.

Q26. How does service discovery work in Kubernetes?

Every Service gets a stable ClusterIP and a DNS name. Pods discover services by DNS name resolution via CoreDNS, and kube-proxy load-balances the ClusterIP to the current set of healthy pod IPs (tracked in EndpointSlices). Environment variables for services are also injected, but DNS is the standard mechanism. As pods come and go, EndpointSlices update automatically.

Q27. What is CNI and how does pod-to-pod networking happen across nodes?

CNI (Container Network Interface) is the plugin standard the kubelet calls to wire up each pod's network. The plugin (Calico, Cilium, Flannel, AWS VPC CNI) assigns each pod a routable IP and ensures every pod can reach every other pod without NAT. Cross-node traffic is carried either by an overlay (VXLAN/IPIP encapsulation) or by native routing/BGP where the underlying network knows the pod CIDRs. (In AWS VPC CNI, pods get real VPC IPs from ENIs.)

Q28. Explain how CNI plugins work and how cross-node pod communication happens.

When a pod starts, the kubelet invokes the CNI plugin, which creates a veth pair, assigns an IP from the node's pod CIDR, and sets up routes. For cross-node traffic, overlay-mode plugins encapsulate packets (VXLAN over UDP 4789) so they traverse the underlay transparently; routed-mode plugins advertise pod CIDRs via BGP so the network routes natively. The receiving node decapsulates/routes the packet to the destination pod's veth.

Q29. How do you debug intermittent packet loss between pods/services?

Check for CPU throttling/conntrack table exhaustion on nodes (`conntrack -S`), MTU mismatches (common with overlays — set correct MTU), and dropped packets on interfaces (`ifconfig`/`ethtool -S`). Verify CNI health and that NetworkPolicies aren't intermittently blocking. Use `tcpdump`/`mtr` between pods, check for node-level network saturation, and confirm kube-proxy rules are consistent. Intermittent loss often points to conntrack limits or an unhealthy backend behind a Service.

Q30. What are Kubernetes network policies and how do they control pod communication?

NetworkPolicies are namespaced firewall rules that select pods by label and define allowed ingress/egress to/from other pods, namespaces, or IP blocks. By default all traffic is allowed; once a pod is selected by any policy, only explicitly allowed traffic passes (default-deny for that direction). They require a CNI that enforces them (Calico, Cilium). Use them to segment microservices and limit blast radius.

Q31. A pod is Running but returning 503 — how do you debug at network, service and ingress level?

Work from the outside in. At ingress: check the Ingress controller logs and that the backend service/port in the Ingress rule is correct. At service: verify the Service selector matches pod labels and that EndpointSlices contain ready pod IPs (`kubectl get endpoints`) — empty endpoints often mean readiness is failing. At pod: confirm the app listens on the expected port and passes its readiness probe. 503 usually means no healthy backend endpoints behind the service.

Q32. A pod is running but not accessible — what will you check?

Confirm the Service selector matches the pod labels and that endpoints are populated. Check the readiness probe (failing readiness removes the pod from endpoints). Verify the container listens on 0.0.0.0 (not 127.0.0.1) on the right port and that containerPort/targetPort match. Check NetworkPolicies and kube-proxy. For external access, verify the Ingress/LoadBalancer and security groups.

Kubernetes — Troubleshooting Scenarios

Q33. What happens internally when a pod gets OOMKilled, and how do you debug and prevent it?

The container exceeded its memory limit (the cgroup memory.max), so the kernel OOM killer terminates its main process with SIGKILL (exit 137) and the kubelet restarts it. Confirm via ``kubectl describe pod`` (Last State: Terminated, Reason: OOMKilled). Prevent it by setting realistic memory requests/limits, profiling and fixing memory leaks, tuning JVM/heap settings to respect cgroup limits, and adding monitoring/alerts on memory usage approaching limits.

Q34. Pods are getting evicted frequently — what will you check?

Check node resource pressure conditions (``kubectl describe node`` → MemoryPressure/DiskPressure/PIDPressure). Look at which QoS class is being evicted (BestEffort first). Verify pods have proper requests so the scheduler doesn't overcommit nodes, check for memory leaks, disk filling from logs/images, and whether the node is undersized. Consider adding nodes (Cluster Autoscaler) or setting resource requests/limits correctly.

Q35. Your application works in staging but fails in production — how do you systematically debug environment differences?

Compare config and secrets (env vars, ConfigMaps), image tags/digests (ensure identical images, not ``latest``), resource limits, replica counts, and feature flags. Check differences in data volume/scale, network policies, DNS, external dependency endpoints, IAM/permissions, and ingress/TLS. Diff the manifests between environments. Reproduce with production-like data/load in staging. Often it's a missing secret, a tighter resource limit, scale-related behavior, or a permissions difference.

Q36. Your application is slow intermittently — how do you troubleshoot?

Correlate latency with metrics: CPU throttling (cgroup throttling from low CPU limits), memory pressure/GC pauses, dependency latency (DB, cache, downstream APIs), and network. Check HPA scaling events, noisy neighbors, connection pool exhaustion, and DNS lookup latency. Use distributed tracing to find the slow span and APM/profilers. Intermittent slowness frequently traces to CPU limit throttling, GC, or a saturated downstream dependency.

Q37. High latency between services — how do you debug?

Use distributed tracing (Jaeger/Tempo/X-Ray) to localize which hop is slow. Check service-side metrics (CPU throttling, queueing), network (MTU, conntrack, retransmits via `tcpdump/mtr`), DNS resolution time, mTLS/proxy overhead if using a service mesh, and connection pool saturation. Verify endpoints are healthy and traffic isn't being sent to a degraded replica. Look at p99 not just averages.

Q38. How do you debug intermittent pod restarts with no clear logs?

Check ``kubectl describe pod`` for restart reason and last state/exit code (137 = OOM, 143 = SIGTERM). Inspect node-level events (``dmesg``, kubelet logs) for OOM kills or evictions. Review liveness probe config — an overly aggressive probe restarts healthy pods. Check resource limits, look for crashes that flush no logs (set up a sidecar or persistent log shipping), and capture core dumps. Often it's liveness probe failures or silent OOM kills.

Q39. How do you troubleshoot production Kubernetes issues like ImagePull errors and pod eviction?

For ImagePullBackOff: check the image name/tag exists, the registry is reachable, and imagePullSecrets/credentials are correct (`kubectl describe pod`` shows the exact error). For evictions: inspect node pressure and pod QoS/requests as above. General workflow: `kubectl get pods``, `describe``, `logs --previous``, `get events --sort-by=.lastTimestamp``, and node inspection.

Q40. How do you debug a pod that is not starting?

Run `kubectl describe pod`` to see events and the current container state (Pending = scheduling issue; ImagePullBackOff = image/registry; CrashLoopBackOff = app crash; Init: errors = init container). Check `kubectl logs --previous``, verify config/secrets/volumes mount, confirm requests fit on a node, and check init containers completed. Events at the bottom of describe almost always name the root cause.

Kubernetes — Resources, Security & Operations

Q41. What are requests and limits, and what happens if limits are not defined?

A request is the guaranteed minimum the scheduler reserves for a container; a limit is the hard ceiling it can use. CPU over the limit is throttled; memory over the limit gets OOMKilled. With no limits, a container can consume all node resources and starve/evict neighbors, and the pod becomes BestEffort/Burstable QoS, making it a prime eviction target. Always set requests; set limits thoughtfully (especially memory).

Q42. How do you monitor resource usage and prevent memory issues in production?

Use metrics-server for `kubectl top``, and Prometheus + Grafana for historical CPU/memory per pod/node, plus alerts on usage approaching limits and on OOMKills/restarts. Set proper requests/limits, fix leaks, right-size workloads with VPA recommendations, and use HPA for load. For the JVM and similar runtimes, ensure they respect cgroup limits.

Q43. How do you provide read-only access to a developer in Kubernetes (RBAC)?

Create a Role (namespace-scoped) or ClusterRole (cluster-wide) granting only `get``, `list``, and `watch`` verbs on the needed resources, then bind it to the user/group/ServiceAccount with a RoleBinding or ClusterRoleBinding. The built-in `view`` ClusterRole already provides read-only access and can be bound directly. Avoid granting `secrets`` read unless required.

Q44. What are common Kubernetes security measures in EKS?

IAM Roles for Service Accounts (IRSA) for least-privilege pod AWS access, RBAC for cluster permissions, private API endpoint and security groups, network policies for pod segmentation, encrypted secrets (KMS envelope encryption), image scanning and signed images, Pod Security Standards/admission policies, node hardening, audit logging to CloudWatch, and keeping the control plane/nodes patched.

Q45. How do you design zero-downtime deployments for stateful applications in Kubernetes?

Use StatefulSets with rolling updates and PodDisruptionBudgets to limit simultaneous disruptions. Ensure proper readiness probes and graceful shutdown (preStop hooks, SIGTERM handling, connection draining). For databases, do ordered, one-replica-at-a-time updates with replication/quorum maintained, run schema migrations backward-compatibly (expand/contract pattern), and verify each replica rejoins/syncs before proceeding.

Q46. How do you ensure zero-downtime cluster upgrades in Kubernetes?

Upgrade the control plane first (managed services do this for you), then nodes via a surge/rolling strategy: cordon and drain one node at a time respecting PodDisruptionBudgets, let pods reschedule onto new/updated nodes, then proceed. Use a blue-green node group (spin up new nodes on the new version, migrate workloads, retire old nodes) for the safest path. Validate workloads and API deprecations before upgrading.

Docker & Containers

Q47. How does Docker work internally (architecture)?

The Docker CLI talks to the Docker daemon (dockerd) over a REST API. dockerd manages images, networks, and volumes and delegates container lifecycle to containerd, which in turn uses runc (an OCI runtime) to create containers using Linux namespaces (isolation) and cgroups (resource limits). Images are layered, content-addressed filesystems assembled via a union/overlay filesystem at runtime.

Q48. What is the difference between the container runtime and the Docker engine?

The Docker engine is the full platform — CLI, daemon, image building, networking, API. The container runtime is the lower-level component that actually creates and runs containers: containerd (high-level: image/lifecycle management) and runc (low-level: spawns the process with namespaces/cgroups). Kubernetes talks to containerd/CRI-O directly via the CRI and no longer needs the full Docker engine.

Q49. What happens internally when you run `docker run`?

The CLI sends the request to dockerd, which pulls the image if absent, creates a writable container layer on top of the read-only image layers, sets up namespaces (PID, net, mount, etc.) and cgroups, configures networking (veth + bridge by default) and volumes, then hands off to containerd/runc to start the process defined by ENTRYPOINT/CMD. The container runs as long as its main process (PID 1) lives.

Q50. How does Docker layer caching work?

Each Dockerfile instruction creates a layer; Docker caches layers and reuses them on rebuild if the instruction and its inputs are unchanged. The cache invalidates from the first changed instruction onward, so order matters: put rarely-changing steps (installing dependencies) before frequently-changing ones (copying source). Copying package manifests and installing deps before copying the full source maximizes cache hits.

Q51. What are multi-stage Docker builds and what stages do they have?

Multi-stage builds use multiple FROM statements so you can build artifacts in a heavy 'builder' stage (compilers, dev dependencies) and then COPY only the final artifacts into a slim runtime stage. This drastically reduces final image size and attack surface because build tools never ship to production. Example: a Node build stage runs ``npm ci && npm run build``, and the runtime stage copies only ``dist`` and production deps.

Q52. How do you reduce / optimize Docker image size for production?

Use multi-stage builds, a minimal base (distroless, slim, or carefully-used Alpine), combine and clean up in single RUN layers (remove apt caches), use `.dockerignore` to exclude junk, copy only needed artifacts, avoid installing dev/build tools in the final image, and pin/clean package manager caches. Fewer layers and smaller bases mean faster pulls and smaller attack surface.

Q53. What is the difference between CMD and ENTRYPOINT?

ENTRYPOINT defines the executable that always runs; CMD provides default arguments (or a default command if no ENTRYPOINT). When both are set in exec form, CMD's values are passed as arguments to ENTRYPOINT. Anything you append to ``docker run`` overrides CMD but not ENTRYPOINT. Use ENTRYPOINT for the fixed binary and CMD for overridable defaults.

Q54. What is the difference between COPY and ADD in a Dockerfile?

COPY simply copies local files/directories into the image. ADD does the same but additionally can auto-extract local tar archives and fetch remote URLs. Best practice is to use COPY for clarity and predictability, and only use ADD when you specifically need tar auto-extraction (remote URL fetching is discouraged — use curl/wget in a RUN if needed).

Q55. How do you manage environment variables and secrets in Docker?

Env vars: set with ENV in the Dockerfile (build/runtime defaults) or ``-e`/`--env-file`` at runtime. Never bake secrets into image layers (they persist in history). For secrets, use BuildKit ``--secret`` mounts at build time, runtime injection via orchestrator secrets (Kubernetes Secrets, Docker Swarm secrets), or an external secrets manager (Vault, AWS Secrets Manager) fetched at startup. For a PHP+MySQL app, pass DB credentials via runtime env/secret mounts, not in the image.

Q56. What are Docker volumes and how do you use them in production?

Volumes are Docker-managed persistent storage that lives outside the container's writable layer, so data survives container restarts/recreation. Named volumes are preferred over bind mounts in production for portability and driver support (e.g., network/cloud volume drivers). Use them for databases, uploaded files, and any stateful data; the container layer should stay ephemeral.

Q57. What is Docker networking and how does it work?

Docker provides network drivers: bridge (default, NAT'd private network per host), host (share the host's network namespace), none (no networking), overlay (multi-host networking for Swarm), and macvlan. Containers on the same user-defined bridge can resolve each other by name via Docker's embedded DNS. Published ports (``-p``) DNAT host ports to container ports.

Q58. How do you secure Docker containers and what are common security risks?

Run as a non-root user, use read-only root filesystems, drop Linux capabilities, set resource limits, use minimal/distroless base images, scan images for CVEs, sign images, and never embed

secrets. Risks: running as root, outdated/vulnerable base images, exposed Docker socket, privileged containers, leaked secrets in layers, and unrestricted egress. Use seccomp/AppArmor profiles and avoid `--privileged`.

Q59. How do you troubleshoot/debug a container that crashes or keeps restarting?

Check `docker logs <container>` (and `--previous` equivalents), inspect exit code with `docker inspect`, and review the restart policy. Run the image interactively (`docker run -it --entrypoint sh image`) to inspect, check for missing config/env, OOM (exit 137), failing healthcheck, or a misconfigured command. For exited containers, `docker logs` plus `docker inspect --format '{{.State.ExitCode}}'` pinpoints the cause.

Q60. What is Docker Compose and how do you use it in real projects?

Compose defines and runs multi-container apps from a single declarative YAML file (services, networks, volumes, dependencies). `docker compose up` starts the whole stack with one command. It's used for local development and integration testing — e.g., app + database + cache wired together — and for simple single-host deployments. For production orchestration at scale you move to Kubernetes.

Q61. How do you scan Docker images for vulnerabilities?

Use scanners like Trivy, Grype, Clair, or registry-integrated scanning (ECR scan on push, Docker Scout). Integrate the scan into CI so builds fail on high/critical CVEs, scan both OS packages and app dependencies, and rescan running images periodically since new CVEs are disclosed over time. Combine with image signing and a minimal base to reduce findings.

Q62. Why avoid Alpine images sometimes, and what are alternatives?

Alpine uses musl libc instead of glibc, which can cause subtle bugs, slower DNS resolution, and compatibility/performance issues for some apps (notably Python wheels and certain compiled binaries), plus harder debugging. Alternatives: `-slim` Debian/Ubuntu variants for compatibility, or distroless images for a minimal, secure runtime without a shell/package manager.

Q63. How do you integrate Docker with CI/CD pipelines?

In CI, build the image (ideally with BuildKit and layer caching), run unit/integration tests, scan for vulnerabilities, tag with the commit SHA, and push to a registry. In CD, deploy that immutable, digest-pinned image to the target (Kubernetes/ECS). Use ephemeral build agents, cache layers between runs, and never use mutable `latest` tags for deploys.

CI/CD & Jenkins

Q64. How do you design a CI/CD pipeline?

Stages typically: checkout → build → unit tests → static analysis/security scans (SAST, SCA, secret scanning) → build & scan image → push artifact → deploy to staging → integration/smoke tests → approval → deploy to production → post-deploy verification. Make it pipeline-as-code, fail fast, keep artifacts immutable and digest-pinned, and add rollback and notifications. Trigger on git push/PR via webhooks.

Q65. What makes a CI/CD pipeline 'enterprise-grade' in large-scale systems?

Pipeline-as-code with reusable shared libraries/templates, security gates (SAST/DAST/SCA/secret scanning/image scanning), policy-as-code, artifact immutability and provenance (signing, SBOMs), environment promotion with approvals, automated rollback, observability of the pipeline itself, ephemeral isolated build agents, secrets managed externally, parallelism/caching for speed, audit trails, and multi-tenancy/RBAC. Reliability and compliance matter as much as speed.

Q66. A CI/CD pipeline suddenly fails after months of stability — how do you troubleshoot?

Compare against the last successful run: what changed externally? Common culprits are a floating dependency/base-image/tool version that auto-updated, an expired credential/certificate/token, a registry or repo outage, agent disk/resource exhaustion, or a deprecated API. Read the failing stage logs, reproduce locally, pin versions, and check for environmental drift on the agents. Pin dependencies to prevent recurrence.

Q67. A CI pipeline builds 50 images and takes 20 minutes — how do you reduce it to under 5 minutes?

Parallelize independent builds across agents, enable BuildKit and aggressive layer caching (and a remote cache), build only changed images (path-based/affected detection in monorepos), use smaller base images, cache dependencies, reuse warm/ephemeral agents, and offload to a build farm or remote builders. Multi-stage builds and dependency pre-baking also cut time significantly.

Q68. How do you reduce build time in general / what if it still takes ~1.5 hours?

Profile the pipeline to find the bottleneck stage. Add dependency and layer caching, parallelize tests and builds, split monolithic builds and only rebuild affected modules, use incremental compilation, scale out agents, move slow integration tests to a separate gated stage, and cache artifacts. If a single compile dominates, invest in build-system caching (e.g., remote build cache) rather than more hardware.

Q69. What is pipeline-as-code and how do you implement it?

Pipeline-as-code means the pipeline definition lives in version control alongside the app (e.g., a Jenkinsfile, GitHub Actions workflow, GitLab CI YAML). It's reviewed, versioned, and reproducible. In Jenkins you implement it with a declarative or scripted Jenkinsfile in the repo, using Multibranch Pipeline so branches/PRs get pipelines automatically.

Q70. What is Jenkins master-agent architecture and how does it work?

The Jenkins controller (master) schedules jobs, stores config, and serves the UI but ideally runs no builds itself. Agents (nodes) connect to the controller and actually execute build steps, isolating workloads and enabling horizontal scale. Agents can be static VMs or, better, ephemeral/dynamic (Kubernetes pods, cloud instances) created per build and destroyed afterward for clean, isolated environments.

Q71. What are the benefits of ephemeral agents, and how do you handle their creation, connection and destruction?

Ephemeral agents give clean, reproducible, isolated build environments, eliminate state leakage between builds, and scale on demand to cut cost. With the Kubernetes plugin, Jenkins creates a pod per build from a pod template, the agent connects back to the controller (JNLP/inbound), runs

the build, and the pod is torn down automatically when the build ends. Same pattern with cloud plugins for EC2 spot agents.

Q72. How do you manage credentials securely in Jenkins?

Use the Jenkins Credentials store (encrypted on the controller) and reference them via credential IDs in pipelines, ideally through the ``withCredentials`` block so secrets are masked in logs. For stronger security, integrate an external secrets manager (HashiCorp Vault, AWS Secrets Manager) and fetch secrets at runtime. Scope credentials to folders, never hardcode them in Jenkinsfiles, and rotate regularly.

Q73. How do you use shared libraries in Jenkins?

Shared libraries are reusable Groovy code (a separate repo with a ``vars/`` and ``src/`` structure) loaded into pipelines via ``@Library``. They let you centralize common steps (build, test, deploy, notifications) so every team's Jenkinsfile stays thin and consistent. Update the library once and all pipelines inherit the change — key for standardizing enterprise pipelines.

Q74. How do you handle parallel execution and pipeline failures in Jenkins?

Use the ``parallel`` step to run independent stages concurrently (e.g., test shards, multi-arch builds). Handle failures with ``try/catch``, ``post { failure { } }` blocks, retries (``retry``), timeouts (``timeout``), and ``catchError`/`unstable`` to control build result. Notify on failure and implement rollback in a post stage. ``failFast true`` aborts siblings when one parallel branch fails.

Q75. How do you implement a rollback strategy in a Jenkins pipeline?

Keep immutable, versioned artifacts/images so any previous version can be redeployed. On failed post-deploy verification, automatically redeploy the last known-good version (or shift traffic back in blue-green). Tie rollback to a parameterized job or a post-failure stage, and for Kubernetes use ``kubectl rollout undo``. With GitOps, rollback is a git revert.

Q76. What are common Jenkins production issues and best practices?

Common issues: controller running builds (resource exhaustion), unbounded build history filling disk, plugin version conflicts, flaky/static agents, secrets in logs, and slow pipelines. Best practices: run builds only on agents (prefer ephemeral), pipeline-as-code, manage plugins deliberately and test upgrades, back up JENKINS_HOME, externalize secrets, set log/artifact rotation, use folders + RBAC, and monitor controller/agent health.

Terraform & Infrastructure as Code

Q77. What is the difference between ``terraform import`` and ``terraform taint``?

``terraform import`` brings an existing real-world resource under Terraform management by writing it into state (you still must write matching config). ``terraform taint`` (now largely replaced by ``terraform apply -replace=...``) marks a managed resource for destruction and recreation on the next apply. Import adopts existing infra; taint/replace forces a resource to be rebuilt.

Q78. How do you manage secrets in Terraform without hardcoding them?

Never put secrets in .tf files or commit them. Pull them at runtime from a secrets manager (Vault, AWS Secrets Manager/SSM Parameter Store) via data sources, pass them as sensitive variables

from CI/environment, mark variables/outputs ``sensitive = true``, and encrypt remote state (which stores secret values) with KMS plus tight access control. Better still, have the app fetch secrets directly so they never enter state.

Q79. What is the difference between ``count`` and ``for_each``? Give a real-world use case.

``count`` creates N indexed copies (addressed by integer index) — good for identical resources where order/count is all that matters. ``for_each`` iterates a map/set and addresses resources by key — stable identity that survives changes in the middle of the collection. Use ``count`` for, say, 3 identical worker nodes; use ``for_each`` to create one IAM user per username in a map, so removing one user doesn't reindex and recreate the others.

Q80. How do you handle drift detection in Terraform?

Drift is when real infrastructure diverges from state (someone changed it manually in the console). Detect it by running ``terraform plan`` (or ``terraform plan -refresh-only``) regularly — ideally on a schedule in CI — which reconciles state with reality and shows differences. Remediate by re-applying to restore desired config, and prevent recurrence by restricting console write access (IAM) and enforcing all changes through the IaC pipeline.

Q81. How do you handle infrastructure drift across multiple teams?

Enforce IaC-only changes via IAM/SCPs that deny manual console writes, run scheduled drift-detection plans per workspace/state, split state by team/component to limit blast radius, use a remote backend with locking, code review via PRs, and policy-as-code (Sentinel/OPA) gates. Surface drift alerts to the owning team and require remediation through the pipeline.

Q82. What is a Terraform remote backend and why is it important?

A remote backend (S3+DynamoDB, Terraform Cloud, etc.) stores the state file centrally instead of locally. It enables team collaboration (shared single source of truth), state locking (prevents concurrent applies from corrupting state), encryption at rest, versioning/recovery, and keeps secrets out of local machines. It's essential for any team or CI-driven workflow.

Q83. How does Terraform state locking work, and what would you do if the lock is stuck?

When using a backend that supports locking (e.g., S3 with DynamoDB), Terraform acquires a lock before write operations so two applies can't run simultaneously and corrupt state. If a process dies mid-apply the lock can get stuck; you first confirm no apply is actually running, then release it with ``terraform force-unlock <LOCK_ID>`` (or delete the lock item in DynamoDB). Be cautious — force-unlocking during a live apply can corrupt state.

Q84. A Terraform apply partially failed and resources are in an inconsistent state — how do you recover safely?

Don't panic-destroy. Re-run ``terraform plan`` to see what Terraform thinks exists vs reality; Terraform records successfully-created resources in state, so a re-apply often completes the rest. Reconcile any resources created but not recorded (import them) or orphaned (remove from state or destroy). Use ``-target`` to fix specific resources, and rely on state versioning/backups in the remote backend to roll back if needed.

Q85. How do you safely roll back infrastructure changes after a failed deployment?

Infrastructure rollback is reverting to the previous known-good configuration: ``git revert`` the change and re-apply, or restore a previous state version from the backend. Because some resource changes are destructive/irreversible (e.g., deleting a database), prefer forward-fixes and design for it — use `create_before_destroy`, blue-green infra, and backups. Always review the plan before applying a rollback.

Q86. What is the difference between ``terraform refresh`` and ``terraform plan``?

``terraform refresh`` (now folded into `plan/apply` as a refresh step) updates the state file to match real-world resources without changing infrastructure. ``terraform plan`` refreshes state and then computes the diff between desired config and current state, showing what would change on apply. Refresh only syncs state; plan tells you the actions Terraform intends to take.

Q87. What is the difference between `local-exec` and `remote-exec` provisioners?

``local-exec`` runs a command on the machine running Terraform (e.g., the CI runner) after a resource is created. ``remote-exec`` runs commands on the newly-created remote resource over SSH/WinRM. Both are a last resort — provisioners are discouraged because they're not idempotent and break the declarative model; prefer `cloud-init/user_data`, configuration management, or baked images.

Q88. How do you manage multiple environments (dev/staging/prod) in Terraform?

Common approaches: separate state per environment via directory-per-env with shared modules (clearest, recommended for prod), or Terraform workspaces with environment-specific tfvars (lighter but easy to mis-target). Keep a reusable module layer and thin per-env root configs that pass different variables/backends. Isolating state per env limits blast radius and avoids accidental cross-env changes.

Q89. Where do you use Terraform workspaces and what is their exact use?

Workspaces let one configuration maintain multiple independent state instances (e.g., dev/qa/prod) using the same code, switching via ``terraform workspace select``. They're handy for ephemeral or parallel environments and feature branches. The caveat: all workspaces share one backend/config, so for strongly-isolated production it's often safer to use separate state/directories than workspaces.

Q90. If you delete the prod workspace but state is stored in an S3 backend, can you bring that state back into a new workspace?

Deleting a workspace deletes its state object. If versioning is enabled on the S3 bucket, you can recover the prior state version and restore it. Otherwise, create a new workspace and rebuild state by importing the existing real resources (``terraform import``) so Terraform re-adopts them without recreating. If neither versioning nor the resources exist, the state is lost and must be reconstructed via import.

Q91. What happens when a provider is upgraded across a major version (e.g., AWS 7.5 to 7.6) — how do you plan for it?

Major provider upgrades can introduce breaking schema changes and behavioral differences. Pin the provider version, read the upgrade guide/changelog, bump the version in a branch, run ``terraform init -upgrade`` and ``terraform plan`` in a non-prod workspace to surface diffs, adjust

deprecated arguments, and test thoroughly before promoting to prod. Apply to dev → staging → prod with plan review at each step.

Q92. A data volume defined in Terraform is being resized from 500 GB to 750 GB — what happens on plan and apply?

If the resource supports in-place modification (e.g., AWS EBS volume size, which can grow without replacement), `plan` shows an in-place update and `apply` expands the volume with no destroy — but the OS filesystem still needs to be extended (`resize2fs/xfs_growfs`) on the instance to use the new space. If the attribute forced replacement, Terraform would destroy and recreate the volume (data loss), so always check the plan for replace vs update.

Q93. How does the Terraform dependency graph (DAG) work internally?

Terraform parses configuration and builds a directed acyclic graph of resources based on explicit references and `depends_on`. It walks the graph to determine order, creating/updating independent resources in parallel and dependent ones in sequence. On destroy it walks the graph in reverse. This DAG is what lets Terraform safely parallelize while respecting dependencies.

Q94. How do you write reusable Terraform modules?

Encapsulate a logical component (VPC, EKS cluster) into a module with well-defined input variables, sensible defaults, and outputs; avoid hardcoding; keep it provider-version-pinned and documented. Compose root configs from modules, version them (Git tags / registry), and keep them small and single-purpose. Pass environment differences as variables rather than forking the module.

Q95. What is the Terraform state file (tfstate)?

The state file is Terraform's record of the real-world resources it manages, mapping config to actual resource IDs and storing attributes/metadata. It's how Terraform knows what exists, computes plans/diffs, and tracks dependencies. It can contain secrets, so it must be stored in a secure, encrypted, locked remote backend — never committed to Git.

Q96. How do you import an existing AWS VPC (or other resource) into Terraform?

Write the matching resource block in config, then run `terraform import aws_vpc.main vpc-0abc123` to bind the real VPC to that address in state. Run `terraform plan` and reconcile the config until there's no diff (adjust attributes to match reality). Newer Terraform supports declarative `import {}` blocks and even config generation. Repeat for dependent resources (subnets, route tables).

Q97. How do you do cross-account provisioning using Terraform?

Define multiple AWS providers with aliases, each assuming a role in a different account (`assume_role` with the target account's role ARN), then set `provider = aws.accountB` on resources/modules that belong to that account. The executing principal needs `sts:AssumeRole` permission into each target account's role. This lets one Terraform run manage resources across accounts.

Q98. How do you handle multiple cloud providers in a single Terraform configuration?

Declare each provider (aws, google, azure) in the same config and reference them per-resource (with aliases if multiple instances of one provider). Terraform builds a single dependency graph across them, so you can, for example, create a DNS record in one cloud pointing at a resource in another. Keep credentials for each provider supplied via environment/secret injection.

AWS & Cloud

Q99. What is the difference between an Internet Gateway and a NAT Gateway, and why is the NAT Gateway placed in a public subnet?

An Internet Gateway (IGW) allows two-way internet connectivity for resources with public IPs in public subnets. A NAT Gateway lets resources in private subnets initiate outbound internet connections (updates, API calls) while blocking unsolicited inbound traffic. The NAT Gateway sits in a public subnet because it itself needs a route to the IGW and a public/elastic IP to reach the internet on behalf of the private instances.

Q100. How does traffic flow from a private subnet to the internet?

A private instance sends outbound traffic to its subnet's route table, which routes 0.0.0.0/0 to the NAT Gateway in a public subnet. The NAT Gateway translates the source to its own public IP and forwards the request through the Internet Gateway to the internet; return traffic comes back to the NAT Gateway, which translates it back to the private instance. Unsolicited inbound connections are not allowed in.

Q101. What are alternatives to a NAT Gateway for private subnet access?

VPC Endpoints (Gateway endpoints for S3/DynamoDB, Interface/PrivateLink endpoints for other AWS services) let private resources reach AWS services without internet egress. A self-managed NAT instance is a cheaper (less reliable) alternative. For controlled outbound, an egress-only proxy or a centralized egress VPC with shared NAT via Transit Gateway reduces cost. For IPv6, an egress-only internet gateway.

Q102. What is a VPC and how does routing work? How is a private subnet identified vs a public one?

A VPC is a logically isolated virtual network with a CIDR block, divided into subnets across AZs. Each subnet has a route table controlling traffic flow. A subnet is public if its route table has a default route (0.0.0.0/0) pointing to an Internet Gateway; it's private if it has no IGW route (typically routing egress to a NAT Gateway instead). So you identify public vs private by inspecting the associated route table's default route.

Q103. What is the difference between Private VPC and Public VPC?

There's no distinct 'private VPC' resource type — the distinction is about subnet configuration. A 'public VPC/subnet' has an Internet Gateway route and resources with public IPs reachable from the internet; a 'private VPC/subnet' has no direct internet route inbound, reaching out only via NAT or VPC endpoints. The same VPC usually contains both public and private subnets.

Q104. What is the difference between Security Groups and NACLs?

Security Groups are stateful, instance/ENI-level firewalls — return traffic is automatically allowed, and they support only allow rules. Network ACLs are stateless, subnet-level — you must explicitly allow both inbound and outbound (return) traffic, and they support both allow and deny rules, evaluated by rule number. SGs are the primary control; NACLs add a coarse subnet-level layer (e.g., blocking specific IPs).

Q105. What happens if a route table is misconfigured?

Traffic goes to the wrong target or nowhere. For example, a private subnet missing its NAT route can't reach the internet; a public subnet missing the IGW route becomes effectively private; an overlapping or wrong CIDR route can blackhole or misroute traffic between subnets/peers. Symptoms are connectivity failures or timeouts; you debug by tracing the route table associated with the source subnet.

Q106. Why can a private EC2 instance not access the internet — what could be wrong?

Check, in order: the subnet route table has 0.0.0.0/0 → NAT Gateway; the NAT Gateway is in a public subnet with an IGW route and a healthy Elastic IP; the instance's security group allows outbound; the NACLs allow outbound and the ephemeral-port return traffic; and DNS resolution works. A missing NAT route or a NAT in a private subnet are the most common causes.

Q107. What is the difference between Multi-AZ and Read Replicas in AWS RDS?

Multi-AZ is for high availability/disaster recovery: a synchronous standby in another AZ that automatically fails over; the standby is not readable. Read Replicas are for read scaling: asynchronous copies you can direct read traffic to, reducing load on the primary. Multi-AZ protects availability; Read Replicas improve read throughput. You can combine both.

Q108. How do you transfer data from an S3 bucket in one AWS account to another (cross-account S3 access)?

Set up cross-account access via either a bucket policy on the source bucket granting the destination account/role access, plus an IAM policy on the accessing role (Account B), or use S3 Cross-Region/same-account replication, or `aws s3 cp/sync` from a role that's allowed by both the bucket policy and its own IAM policy. Ensure object ownership/ACLs (or bucket-owner-enforced) are handled so the destination can read the objects.

Q109. How do you provide access to an S3 bucket (the two ways)?

Identity-based: attach an IAM policy to the user/role granting S3 actions on the bucket/objects. Resource-based: attach a bucket policy to the bucket granting a principal access. For cross-account, you typically need both sides to allow it. Prefer IAM roles and least-privilege; use VPC endpoints and block public access unless explicitly required.

Q110. How would you increase an EC2 volume size without downtime (e.g., 50 GB disk that's 48 GB full)?

EBS supports online elastic volume resize: modify the volume size in the console/CLI (`aws ec2 modify-volume`), wait for it to enter the 'optimizing' state, then extend the partition (`growpart`) and grow the filesystem online (`resize2fs` for ext4, `xfs_growfs` for XFS) — all without stopping the instance. No reboot or downtime is required.

Q111. How do you troubleshoot disk space issues / extend a block volume that's full?

Find usage with ``df -h`` (filesystem) and ``du -sh *`` (directories); clear logs/temp/old artifacts and rotate logs. If genuinely full, resize the EBS volume online as above and grow the filesystem. Check for deleted-but-open files (``ls -l | grep deleted``) holding space, and inode exhaustion (``df -i``).

Q112. What is Auto Scaling — explain min, max and desired capacity. How do you handle a sudden traffic spike?

An Auto Scaling Group keeps the number of instances between ``min`` and ``max``, targeting ``desired`` capacity; scaling policies (target tracking on CPU/requests, step, or scheduled) adjust desired within those bounds. For a sudden spike: front with a load balancer, use target-tracking/predictive scaling, ensure warm capacity/headroom, cache and use CDN, and combine with application-level autoscaling. Min guarantees baseline, max caps cost/blast radius.

Q113. How do you design a highly available application across multiple regions?

Run the stack in at least two regions, each multi-AZ. Use Route 53 latency/failover/geolocation routing or a global accelerator to direct users, replicate data (cross-region DB replicas, S3 replication, DynamoDB global tables), keep infrastructure as code for parity, and design stateless app tiers behind regional load balancers. Decide active-active vs active-passive based on RTO/RPO and data-consistency needs, and test failover regularly.

Q114. What is the difference between EBS and EFS, and what happens to EBS on EC2 termination?

EBS is block storage attached to a single instance (usually one AZ); EFS is a managed NFS shared file system multiple instances/AZs can mount concurrently. By default the root EBS volume is deleted on instance termination (`DeleteOnTermination=true`), while additional data volumes can be set to persist. EFS persists independently of any instance.

Q115. How do you postpone a scheduled EC2 maintenance event?

For events that involve a reboot/retirement, you can often act before the deadline: for instances backed by EBS, stop and start the instance (which moves it to healthy hardware) to clear a scheduled reboot/retirement, or proactively migrate. AWS doesn't let you arbitrarily 'snooze' the notification, but stopping/starting (or replacing) the instance before the window addresses the underlying maintenance on your schedule.

Q116. How do you do geo-location based routing using AWS?

Use Route 53 geolocation (or geoproximity) routing policies to return different endpoints based on the user's geographic origin — directing EU users to EU infrastructure, etc. Combine with CloudFront for edge caching and latency-based routing for performance. Geolocation is useful for data residency/compliance and localized content.

Q117. What are the different ways to host a Docker container in AWS?

ECS on EC2 (you manage instances), ECS on Fargate (serverless containers), EKS (managed Kubernetes) on EC2 or Fargate, App Runner (fully managed for web apps/APIs), Lambda with container images (for event-driven/short tasks), and plain EC2 running Docker yourself. Choice depends on control vs operational overhead, scale, and workload type.

Q118. Can you deploy a Docker container on AWS Lambda, and what are the size/memory limits?

Yes — Lambda supports container images up to 10 GB as the deployment package. Memory is configurable from 128 MB up to 10,240 MB (10 GB), with CPU scaling proportionally to memory. The /tmp ephemeral storage can be up to 10 GB, and functions have a 15-minute max execution timeout. It's suited to event-driven, bursty, short-running workloads rather than long-running services.

Q119. What is a layer in AWS Lambda?

A Lambda layer is a packaged archive of libraries, dependencies, or custom runtimes that can be shared across multiple functions, so you don't bundle the same dependencies into every function. Layers reduce deployment package size and centralize common code/dependencies. A function can use up to five layers, and they're mounted into /opt at runtime. (Layers apply to zip-based functions, not container-image functions.)

Q120. What are S3 lifecycle policies for cost optimization?

Lifecycle rules automatically transition objects to cheaper storage classes over time (Standard → Standard-IA → Glacier/Deep Archive) and expire/delete old objects or incomplete multipart uploads. They cut storage cost for data with predictable access patterns. S3 Intelligent-Tiering can auto-optimize when access patterns are unknown.

Q121. What cost optimization initiatives have you taken / how do you optimize cloud cost?

Right-size instances, use Spot for fault-tolerant/batch workloads, Savings Plans/Reserved Instances for steady baselines, autoscaling to match demand, S3 lifecycle policies and storage-class tiering, delete unattached EBS/old snapshots/idle resources, use VPC endpoints to cut NAT/data-transfer costs, consolidate NAT gateways, set budgets/alerts, and analyze with Cost Explorer/CUR. Tagging enables per-team cost accountability.

Q122. Your AWS bill suddenly increased 3x overnight — how do you investigate and optimize?

Open Cost Explorer and filter by service, region, and usage type to find the spike; check the Cost and Usage Report for the exact line items. Common causes: data-transfer/NAT egress surge, a runaway autoscaling group or Lambda loop, accidental large EC2/GPU instances, S3 request/Glacier-retrieval spikes, forgotten resources, or a leaked credential mining crypto. Set up anomaly detection and budgets, then remediate the specific driver and add guardrails (SCPs, budget alerts).

Q123. How do you design a 3-tier architecture using Infrastructure as Code?

Three tiers — web/presentation, application/logic, and data — across a VPC. Public subnets host an ALB and bastion; private app subnets host the application tier (ASG/EKS) reachable only from the ALB; isolated data subnets host RDS/cache reachable only from the app tier. Codify VPC, subnets, route tables, SGs, ALB, ASG/EKS, and RDS in Terraform modules, with least-privilege security groups between tiers and multi-AZ for HA.

Networking & System Design

Q124. What is the difference between a Reverse Proxy, an API Gateway, and a Load Balancer?

All three sit between clients and servers but solve different problems. A reverse proxy decides where a request goes — hiding/protecting backends, TLS termination, caching, routing. An API gateway decides whether and how a request is allowed — authentication, rate limiting, request transformation, API versioning — a control plane for APIs. A load balancer decides which instance of the same service handles a request — health checks, traffic distribution, failover. They overlap (NGINX/Envoy can be all three); the difference is how you use it.

Q125. How does AWS Load Balancer route traffic / what types of load balancers exist?

AWS offers the ALB (L7, HTTP/HTTPS, routes by host/path/header to target groups), NLB (L4, TCP/UDP, ultra-high performance and static IPs), and the legacy CLB. An ALB evaluates listener rules to pick a target group, then sends the request to a healthy registered target using its algorithm (round robin / least outstanding requests), honoring health checks and optionally sticky sessions.

Q126. What happens internally when you hit a CloudFront URL?

DNS resolves the CloudFront domain to the nearest edge location (PoP). The edge checks its cache for the requested object; on a hit it returns it immediately, on a miss it fetches from the origin (S3/ALB/custom) — often over the optimized AWS backbone — caches it per the cache policy/TTL, and returns it. TLS terminates at the edge, and features like WAF, signed URLs, and Lambda@Edge can run there.

Q127. How does DNS resolution work step by step?

The resolver checks local/browser/OS cache; on a miss it queries the configured recursive resolver. The recursive resolver, if uncached, asks a root server (which points to the TLD servers), then the TLD server (e.g., .com) points to the domain's authoritative name servers, which return the final record (e.g., A/AAAA). The answer is cached per its TTL at each layer and returned to the client.

Q128. What are Route 53 routing policies?

Simple (single record), weighted (split traffic by percentage, good for canaries), latency-based (route to the lowest-latency region), failover (primary/secondary with health checks), geolocation (by user location, for compliance/localization), geoproximity (bias by geography/resources), and multivalue answer (return multiple healthy records). Health checks underpin failover and multivalue.

Q129. How do you check open ports, find large files, and find recently modified files in Linux?

Open ports: `ss -tulnp` (or `netstat -tulnp`). Large files: `du -ah / | sort -rh | head` or `find / -type f -size +100M`. Files modified in the last 30 minutes: `find /path -type f -mmin -30`. For processes/file handles use `lssof`, and `df -h`/`df -i` for disk and inode usage.

Q130. How do you do network connectivity checks between servers (Linux basics, process monitoring, file handling)?

Connectivity: ``ping``, ``traceroute`/`mtr``, ``telnet host port`` or ``nc -zv host port`` for TCP reachability, ``curl -v`` for HTTP, ``dig`/`nslookup`` for DNS. Process monitoring: ``top`/`htop``, ``ps aux``, ``systemctl status``, ``journalctl``. File handling: ``ls``, ``cp`/`mv`/`rm``, ``cat`/`less`/`tail -f``, ``grep``, ``find``, permissions with ``chmod`/`chown``, and ``ls -l`` for open files.

Observability, Monitoring & Logging

Q131. How do you design an observability stack (metrics, logs, traces) for large-scale systems?

Three pillars: metrics (Prometheus/CloudWatch + Grafana for dashboards and alerting), logs (centralized via EFK/ELK, Loki, or CloudWatch with structured JSON), and traces (OpenTelemetry → Jaeger/Tempo/X-Ray for request-level latency across services). Standardize on OpenTelemetry instrumentation, correlate the three via trace/request IDs, define SLOs and alert on symptoms (latency/error rate) not just resources, and manage cardinality and retention to control cost.

Q132. How do logs, metrics and traces work together in observability?

Metrics tell you something is wrong (error rate/latency spiking) cheaply and over time; traces show where in the request path the problem is (which service/span is slow); logs tell you why (detailed context/errors at that point). Correlate them with a shared trace/request ID so you can pivot from an alerting metric, to the relevant trace, to the exact log lines.

Q133. How do you implement SLO-based alerting without alert fatigue?

Define SLOs (e.g., 99.9% of requests under 300 ms) and an error budget; alert on burn rate of the error budget rather than raw thresholds, using multi-window multi-burn-rate alerts (fast burn = page, slow burn = ticket). Alert on user-facing symptoms, not every resource blip; route by severity, deduplicate, and tie alerts to runbooks. This focuses pages on issues that actually threaten the SLO.

Q134. An application shows latency spikes every 60 seconds (or every morning at market open) — how do you debug?

A precise, periodic spike points to a scheduled/triggered event rather than random load. Check cron jobs, metric scrape intervals, GC cycles, cache TTL expiry/refresh stampedes, log flushing, connection pool recycling, or batch jobs firing on that cadence. For a morning spike, look at cold caches, autoscaling lag at traffic onset, connection storms, or a daily job. Correlate the spike timestamp with scheduler logs and per-component metrics/traces; the periodicity is the biggest clue.

Q135. Grafana shows stale dashboards across clusters after an OpenTelemetry collector upgrade — incident triage chain?

Confirm scope (which dashboards/clusters/data sources), then check whether data ingestion stopped at the collector, the backend (Prometheus/Tempo), or just Grafana's query path. Inspect the upgraded collector's logs/config for breaking changes (renamed metrics, dropped pipelines, auth), verify the collector is exporting and the backend is receiving. Roll back the collector if it's the cause, then fix-forward; communicate status and capture an RCA on the upgrade gap.

Q136. How do you decide which logs should go to Prometheus vs Splunk (or another logging system)?

Prometheus is for numeric time-series metrics, not logs — so aggregate counters/gauges (request rates, error counts, latencies) belong there. Detailed event/audit/debug text logs go to a log system (Splunk/ELK/Loki). The rule: if you need to aggregate/alert on a number over time, make it a metric; if you need searchable event detail/context, make it a log. Extract metrics from logs only when you can't instrument directly.

Q137. How do you handle log isolation and avoid high cardinality issues?

Isolate logs per tenant/service/namespace with labels and separate indices/streams, and apply RBAC and retention per stream. Avoid high cardinality by not putting unbounded values (user IDs, request IDs, full URLs) into metric labels or log index fields — keep those in the log body, not as aggregation keys. Sample verbose logs, cap label sets, and pre-aggregate where possible to protect the backend and control cost.

Q138. Explain the complete EFK (Elasticsearch, Fluentd, Kibana) flow.

Fluentd (or Fluent Bit) runs as a DaemonSet, tails container/node logs, parses and enriches them (adds pod/namespace metadata), and forwards to Elasticsearch, which indexes and stores them. Kibana queries Elasticsearch to provide search, dashboards, and visualization. The flow: app stdout → node log files → Fluentd collect/parse → Elasticsearch index → Kibana visualize.

Q139. How do you manage logs in Kubernetes clusters?

Apps log to stdout/stderr; a node-level agent (Fluent Bit/Fluentd/Vector) DaemonSet collects them along with pod metadata and ships to a backend (Elasticsearch, Loki, CloudWatch). Use structured (JSON) logging, set retention and index lifecycle policies, separate by namespace/team, and avoid logging secrets. Centralization is essential since pods and their local logs are ephemeral.

Q140. How do you use Grafana and integrate it into a project?

Grafana is a visualization/alerting layer that queries data sources (Prometheus, Loki, CloudWatch, Elasticsearch). Integrate by pointing it at your metrics/logs backends, building dashboards per service/SLO, and configuring alert rules with notification channels (Slack/PagerDuty). Provision dashboards and data sources as code (JSON/Terraform) for reproducibility.

Q141. How do you integrate Prometheus into your application?

Instrument the app with a Prometheus client library to expose a `/metrics` endpoint (counters, gauges, histograms), or run an exporter for off-the-shelf software. Prometheus scrapes these endpoints on an interval (discovered via service discovery/ServiceMonitor in Kubernetes), stores the time series, and you alert via Alertmanager and visualize in Grafana. Focus on RED/USE metrics.`

Security & Secrets Management

Q142. How do you design a secure CI/CD pipeline (DevSecOps) and where do you integrate security scans?

Shift security left and gate each stage: secret scanning and SAST on commit/PR, SCA (dependency CVE) scanning at build, container image scanning before push, IaC scanning

(tfsec/Checkov) on infra code, and DAST against staging. Sign artifacts and generate SBOMs, enforce least-privilege pipeline credentials from a secrets manager, use ephemeral isolated agents, and add policy-as-code gates. Fail the build on high/critical findings and keep an audit trail.

Q143. A secret was exposed in GitHub — what is your incident response plan?

Treat it as compromised regardless of how briefly it was exposed. Immediately revoke/rotate the secret at the source (the credential is now burned), then investigate for misuse via access/audit logs and scope the blast radius. Remove it from the repo and history (BFG/filter-repo) but assume it's already harvested. Add prevention: secret scanning (pre-commit + server-side), move secrets to a manager, and short-lived/dynamic credentials. Document a blameless postmortem.

Q144. How do you design secure secrets management across 100+ microservices?

Centralize in a secrets manager (Vault, AWS Secrets Manager) with per-service least-privilege access via identity (IRSA/Vault Kubernetes auth), so no secret lives in code, env files, or images. Prefer short-lived/dynamic secrets (Vault dynamic DB creds), automatic rotation, encryption in transit and at rest, full audit logging, and injection at runtime (sidecar/CSI driver). Namespace policies per service and never share credentials across services.

Q145. How do you securely manage application secrets in Kubernetes?

Avoid plain Kubernetes Secrets alone (only base64). Enable etcd encryption at rest (KMS) and tight RBAC, and prefer an external store: Vault with the Agent Injector or the Secrets Store CSI Driver, or External Secrets Operator syncing from AWS Secrets Manager. Use workload identity (IRSA) for access, rotate regularly, never log secrets, and inject at runtime rather than baking into images.

Q146. CyberArk Conjurer vs AWS Secrets Manager for EKS — which would you choose and why?

It depends on context. AWS Secrets Manager is the simpler, native choice for an AWS/EKS-only footprint: tight IAM/IRSA integration, automatic rotation for AWS services, low operational overhead. CyberArk Conjurer fits when you need a cloud-agnostic, enterprise secrets platform with strong policy, broad integrations, and centralized governance across hybrid/multi-cloud. For pure EKS, Secrets Manager (often via External Secrets Operator); for enterprise multi-cloud governance, Conjurer.

Q147. Your system passed all security scans but still got compromised — what security layers were missing?

Scans catch known CVEs and misconfigurations but not everything. Likely gaps: runtime security (detecting anomalous behavior), network segmentation/least-privilege (over-permissive IAM, flat networks enabling lateral movement), secrets hygiene and rotation, supply-chain integrity (unsigned dependencies/images), business-logic flaws and authn/authz issues scanners miss, missing WAF/rate limiting, and weak monitoring/detection. Defense in depth and zero-trust beyond point-in-time scanning.

Q148. What is a vulnerability and what kinds of security scans do you run on which applications?

A vulnerability is a weakness (in code, dependencies, config, or infrastructure) that an attacker can exploit to compromise confidentiality, integrity, or availability. Scan types: SAST on source code,

SCA on dependencies/libraries, container image scanning on built images, DAST against running web apps/APIs, IaC scanning on Terraform/manifests, and secret scanning across repos. Web apps get SAST+DAST+SCA; containers get image scanning; infra code gets IaC scanning — integrated into CI.

Q149. What is cognitive complexity (in code quality scans)?

Cognitive complexity (a SonarQube metric) measures how hard code is for a human to understand, by penalizing nested control flow, breaks in linear flow, and deeply nested logic — unlike cyclomatic complexity, which just counts paths. High cognitive complexity flags methods that should be refactored/simplified, improving maintainability and reducing bug risk. It's a code-quality gate, not a security one.

Deployment Strategies

Q150. Explain rolling vs blue-green vs canary deployments with real production use cases.

Rolling: gradually replace old pods with new ones — default in Kubernetes, no extra infra, but mixed versions briefly run together (use for routine stateless updates). Blue-green: run two full environments and switch all traffic at once — instant cutover and rollback, but doubles resources (use for risky releases needing fast rollback). Canary: route a small % of traffic to the new version, watch metrics, then ramp up — safest for validating in prod with real traffic (use for high-risk, high-traffic services).

Q151. What is blue-green deployment, and how do you do it in Kubernetes?

Blue-green keeps two identical environments: 'blue' (current) serves traffic while 'green' (new version) is deployed and tested. You switch traffic to green by flipping the Service selector or ingress/load-balancer target, with instant rollback by switching back. In Kubernetes, run two Deployments (blue/green) and update the Service's label selector, or use a controller like Argo Rollouts to orchestrate the cutover.

Q152. What is a rolling update in Kubernetes and how do you roll back?

A rolling update incrementally replaces old pods with new ones controlled by maxSurge/maxUnavailable, keeping the app available throughout. Kubernetes records revision history, so you roll back with ``kubectl rollout undo deployment/<name>`` (optionally ``--to-revision=N``), and monitor with ``kubectl rollout status``. Readiness probes gate each new pod before old ones are removed.

Q153. What is the difference between rolling updates and rollback?

A rolling update is the forward process of deploying a new version pod-by-pod. A rollback is reverting to a previous known-good revision when the new version misbehaves. Kubernetes Deployments keep revision history enabling ``rollout undo``. One ships change; the other reverses it.

Q154. How do you design canary deployments and zero-downtime deployments for critical applications?

Canary: deploy the new version alongside the old, shift a small traffic slice (via service mesh, ingress weights, or Argo Rollouts), automatically analyze metrics (error rate/latency), then

progressively increase or auto-rollback on regression. Zero-downtime overall: readiness probes, graceful shutdown (preStop + SIGTERM handling + connection draining), PodDisruptionBudgets, backward-compatible DB migrations (expand/contract), and surge capacity so capacity never dips during the rollout.

GitOps, ArgoCD & Git

Q155. How does ArgoCD (GitOps) ensure consistency between Git and cluster state?

ArgoCD treats a Git repo as the single source of truth for desired cluster state. It continuously compares the live cluster state against the manifests in Git; when they diverge it reports OutOfSync and either auto-syncs or waits for approval to reconcile. It can self-heal (revert manual cluster changes) and prune resources removed from Git, so the cluster always converges to what's committed. The App-of-Apps pattern manages many apps declaratively.

Q156. How does Git merge differ from rebase internally?

Merge creates a new merge commit that ties two branch histories together, preserving the actual branching history (non-linear). Rebase replays your branch's commits on top of the target branch, creating new commits with new hashes for a linear history. Merge is non-destructive and keeps context; rebase rewrites history (don't rebase shared/pushed branches). Use merge to integrate, rebase to keep a clean linear feature history.

Q157. What is a divergent branch and how do you handle it? What branching strategy do you follow?

Branches diverge when both local and remote have commits the other lacks (common after someone force-pushes or both sides advance). Handle it by fetching and then merging or rebasing onto the remote (or resetting if local should match remote), resolving conflicts deliberately. Strategies: trunk-based development (short-lived branches, frequent merges to main, feature flags) suits CI/CD best; GitFlow (long-lived develop/release/hotfix branches) suits scheduled releases.

Q158. How did you migrate a codebase from one repository/system to another (e.g., Bitbucket to GitHub)?

Mirror-clone the source to preserve all history, branches, and tags (`git clone --mirror`), then push to the new remote (`git push --mirror`). Migrate metadata separately (PRs, issues, webhooks, CI config, branch protections, secrets), update remotes and pipeline integrations, and run the new pipeline to validate. Freeze writes on the old repo during cutover and communicate the switch to the team.

Incident Response, SRE & Reliability

Q159. Production is down and users are impacted — what are your first 5 steps as a DevOps/SRE engineer?

1) Acknowledge and declare the incident, assign an incident commander, open a comms channel and a status page update. 2) Assess scope/impact and severity from dashboards and alerts. 3) Stabilize/mitigate first — roll back the recent change, fail over, scale out, or shed load to restore service (mitigation before root cause). 4) Keep stakeholders updated regularly. 5) Once recovered,

capture timeline and run a blameless postmortem with action items. The priority order is: restore service, then diagnose.

Q160. How do you handle production downtime and incident response?

Detect via monitoring/alerting, triage severity, and assemble responders with clear roles (incident commander, comms, ops). Mitigate to restore service quickly (rollback/failover/scale), communicate transparently to stakeholders/users, then investigate root cause once stable. Follow with a blameless postmortem, track remediation actions, and feed learnings back into runbooks, alerts, and preventative work. Practice with game days.

Q161. What is your approach when a system goes down — how do you approach it?

Stay calm and methodical: confirm the alert is real and scope the impact, check recent changes (deploys/config/infra are the most common cause), look at the four golden signals (latency, traffic, errors, saturation), and form a hypothesis from the strongest signal. Mitigate first (rollback/failover) to stop user pain, then drill into root cause with logs/traces. Communicate throughout and document for the postmortem.

Q162. How do you diagnose performance issues such as high database response times?

Check DB metrics: slow query logs, query plans (missing indexes, full scans), lock contention/blocking, connection pool exhaustion, CPU/IO/memory saturation, and cache hit ratios. On the app side, look for N+1 queries and chatty access patterns. Mitigate with indexing, query optimization, caching, read replicas for read scaling, and connection pooling. Correlate with deploys or traffic changes that triggered it.

Q163. What are post-incident learnings and follow-up improvements?

After recovery, run a blameless postmortem documenting the timeline, impact, root cause, detection/response gaps, and contributing factors. Produce concrete action items with owners and deadlines: add/adjust alerting and runbooks, fix the root cause, add tests/safeguards, improve automation or rollback, and address process gaps. Track the actions to completion and share learnings org-wide so the same failure doesn't recur.

Q164. What strategies do you use to quickly stabilize systems during outages?

Roll back the most recent change first (it's the likeliest trigger), scale out instances/pods to absorb load, shed or rate-limit non-critical traffic, fail over to healthy regions/replicas, enable circuit breakers, flush/warm caches, and isolate the failing component to limit blast radius. The goal is fast mitigation to restore service; deep root-cause analysis comes after stability is regained.

Q165. How do you design systems for graceful degradation during failures?

Design so partial failures don't cascade: circuit breakers and timeouts on dependencies, fallbacks (cached/stale data, default responses), bulkheads to isolate resource pools, rate limiting and load shedding to protect the core, retries with backoff and jitter, and feature flags to disable non-essential features under stress. Prioritize core functionality so the system stays usable even when subsystems fail.

Q166. How do you maintain consistency and quorum during a regional failover for 1,000+ stateful services across multi-region clusters?

Use quorum-based replication (Raft/Paxos systems) with an odd number of regions/zones so a majority survives a regional loss; for databases, run synchronous replication where consistency is critical and accept the latency, or async with clear RPO where you can. Define leader-election and fencing to prevent split-brain, automate controlled failover with health-based promotion, and validate data integrity before redirecting traffic. Regularly run failover drills and reconcile any divergence after recovery.

Q167. How would you design audit-grade logging for all infrastructure actions while keeping overhead minimal and meeting compliance?

Capture every control-plane/infra action from authoritative sources (CloudTrail, Kubernetes audit logs, IaC pipeline logs) into an append-only, tamper-evident store (immutable/WORM, with integrity hashing) with defined retention to meet compliance. Keep overhead low by logging at the API/control-plane layer rather than instrumenting everything, sampling only high-volume non-audit data (never audit events), and shipping asynchronously. Enforce least-privilege access to logs and prove provenance for auditors.

Q168. How do you define 'SLOs that actually matter' in financial systems where failure is a loss, not just an error?

Tie SLOs to business/financial impact, not vanity infra metrics: measure what a failure costs (missed trades, reconciliation errors, settlement delays), set objectives on user-facing correctness and timeliness (e.g., trade submission success, end-to-end latency within market windows), and weight integrity/correctness alongside availability. Define explicit error budgets with stakeholders, account for regulatory consequences, and ensure SLOs drive prioritization of reliability work.

Q169. How do you implement an adaptive backoff + SLA-aware retry policy for a batch job throttled by a downstream API?

Use exponential backoff with jitter to avoid thundering-herd retries, respect the downstream's rate-limit signals (HTTP 429 / Retry-After), and add a circuit breaker. Make it SLA-aware by budgeting total retry time against the remaining SLA window — back off less aggressively if time is tight, or fail fast and alert if the SLA can't be met. Use a token-bucket/concurrency limiter to stay under the downstream's throughput, and queue/prioritize work.

Python (Scripting for DevOps)

Q170. What is the difference between a list and a tuple in Python?

A list is mutable (you can add/remove/change elements) and uses square brackets; a tuple is immutable (fixed after creation) and uses parentheses. Tuples are slightly faster and can be used as dictionary keys or set members (being hashable when their contents are), and they signal that data shouldn't change. Use lists for collections that change, tuples for fixed records/coordinates.

Q171. What is a list comprehension?

A list comprehension is a concise way to build a list in one expression: `[x*2 for x in nums if x > 0]`. It's more readable and usually faster than an equivalent for-loop with append. There are also dict

and set comprehensions and generator expressions for memory-efficient iteration. Keep them simple — deeply nested comprehensions hurt readability.

Q172. Write a Python script to back up (or find/delete) files older than 30 days.

Walk the directory, compare each file's modification time to a cutoff, and act on the old ones:

```
import os, time
cutoff = time.time() - 30*24*60*60
for root, _, files in os.walk('/path/to/dir'):
    for name in files:
        p = os.path.join(root, name)
        if os.path.getmtime(p) < cutoff:
            # shutil.copy2(p, backup_dir) to back up, or os.remove(p) to delete
            print('old file:', p)
```

Use pathlib for cleaner code, and dry-run (print) before deleting in production.